Socket Programming using Python

Tushar B. Kute,
http://tusharkute.com

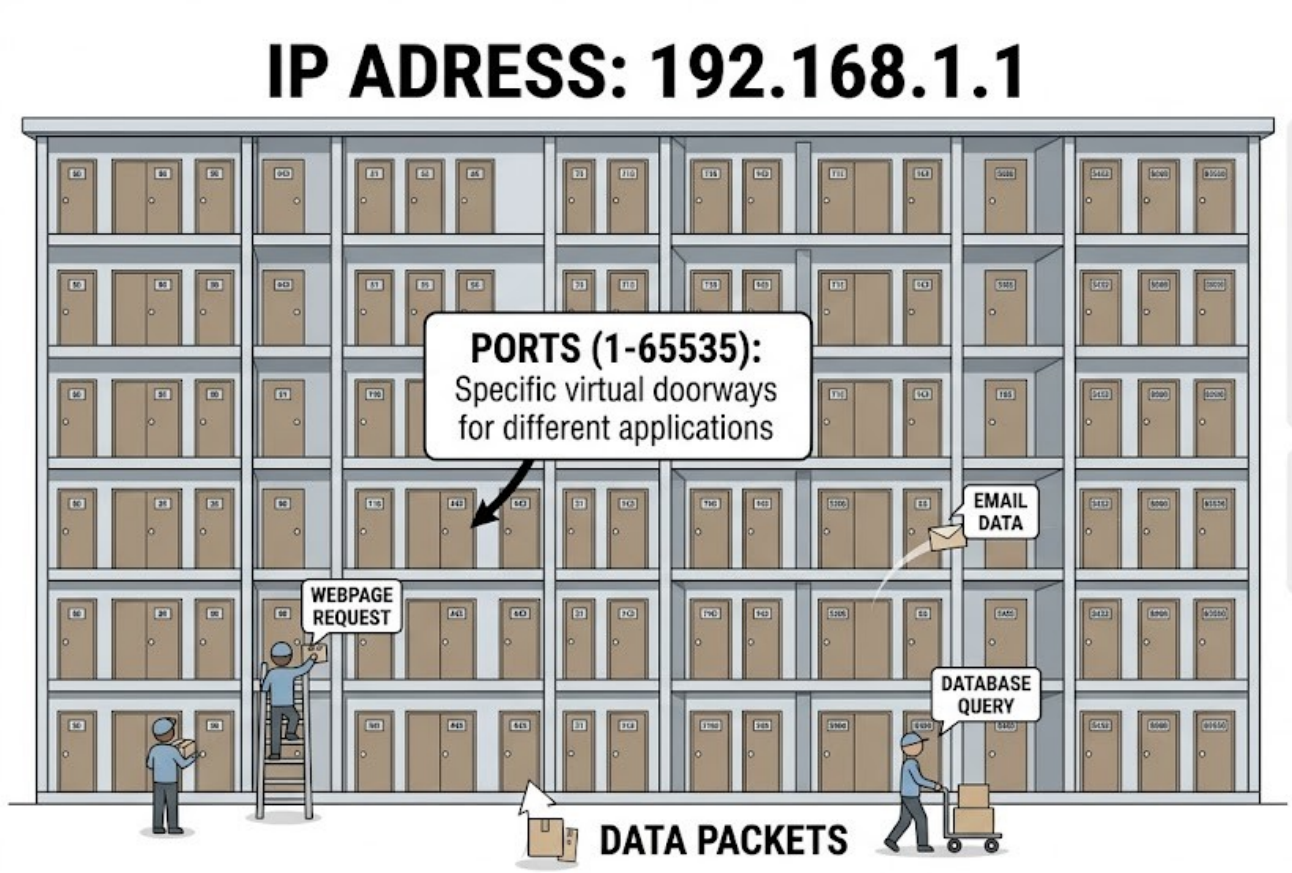
The Foundation of Networking

- A socket is a software endpoint that establishes bidirectional communication between a server and a client program.
- They allow distinct processes to communicate over a network (or on the exact same machine).
- Used for web browsers, chat applications, multiplayer games, and file transfers.

Finding the Machine: IP Addresses

- The unique network address of a computer.
- IPv4 Example: 192.168.1.5
- Localhost (Loopback): 127.0.0.1 — Used when the client and server are running on the exact same computer.

Finding the Application: Ports



IP ADDRESS:
Finds the computer
(the building)

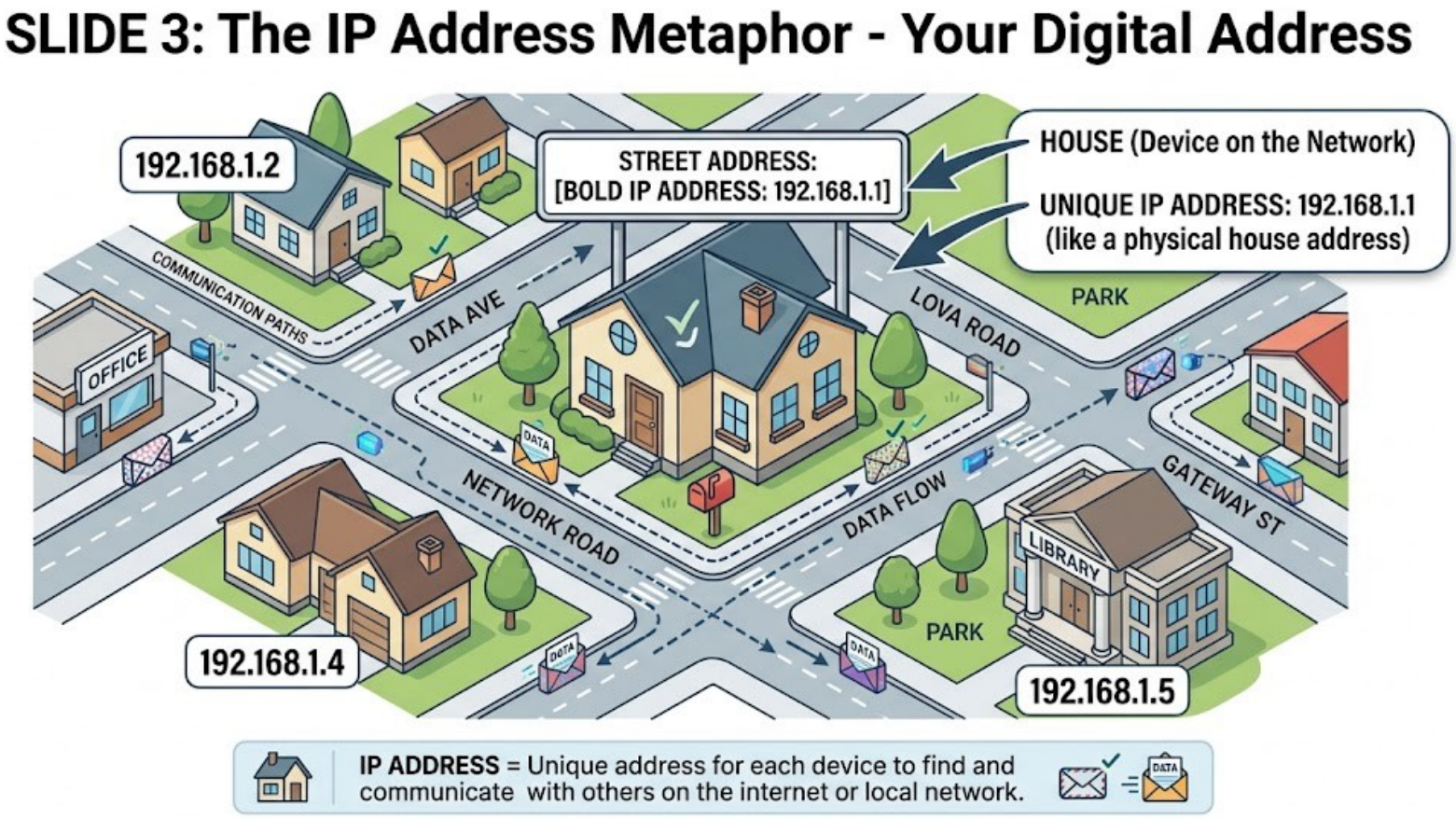
PORT:
Finds the application
(the mailbox/door)

PORT:
Finds the application
(the mailbox/door)

Finding the Application: Ports

- A virtual endpoint inside the computer.
- While the IP finds the computer, the port finds the specific program you want to talk to.
- Range: 0 to 65535.
- Ports under 1024 are reserved (e.g., Port 80 for HTTP web traffic).

Finding the Application: Ports



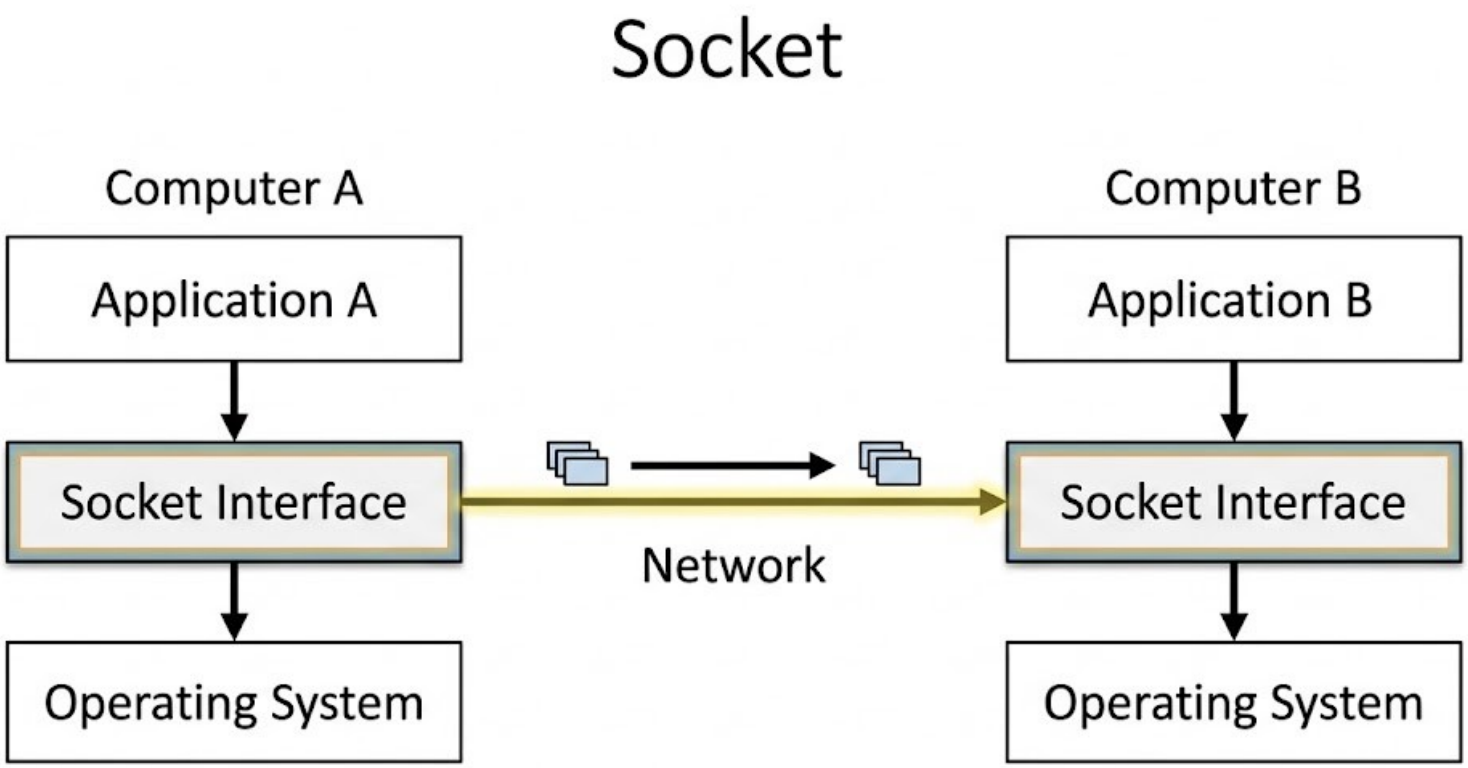
Choosing Your Protocol

- TCP (Transmission Control Protocol): Reliable, ordered, connection-oriented. Guarantees delivery. (e.g., Web browsing, text chat).
- UDP (User Datagram Protocol): Fast, connectionless, unreliable. Packets can drop. (e.g., Live video streaming, fast-paced gaming).

Socket Programming in Python

- Python includes the built-in socket module.
- No pip install required.
- Provides a direct, low-level interface to the operating system's networking API.
  - import socket

Socket Programming in Python



The TCP Server Flow

- The server must be set up and waiting before any client can connect.
- Steps:
  - Create → Bind → Listen → Accept → Receive/Send → Close.

TCP Server: Creation

- AF_INET specifies we are using IPv4 addresses.
- SOCK_STREAM specifies we are using the TCP protocol.

```
HOST = '127.0.0.1'
PORT = 65432
server_socket = socket.socket(socket.AF_INET,
socket.SOCK_STREAM)
```

TCP Server: Binding & Listening

- Bind: Associates the socket with a specific IP and Port.
- Listen: Puts the socket into server mode, waiting for connections.

```
server_socket.bind((HOST, PORT))
server_socket.listen()
print("Listening for connections...")
```

TCP Server: The Blocking accept()

- .accept() halts the program's execution until a client actually connects.
- Returns a new socket object dedicated to that specific client, plus the client's IP address.

```
conn, addr = server_socket.accept()
print(f"Connected by {addr}")
```

TCP Server: Receiving and Sending

- We use an infinite loop to keep the conversation going.
- .recv(1024) reads up to 1024 bytes of data.
- .sendall() echoes it back.

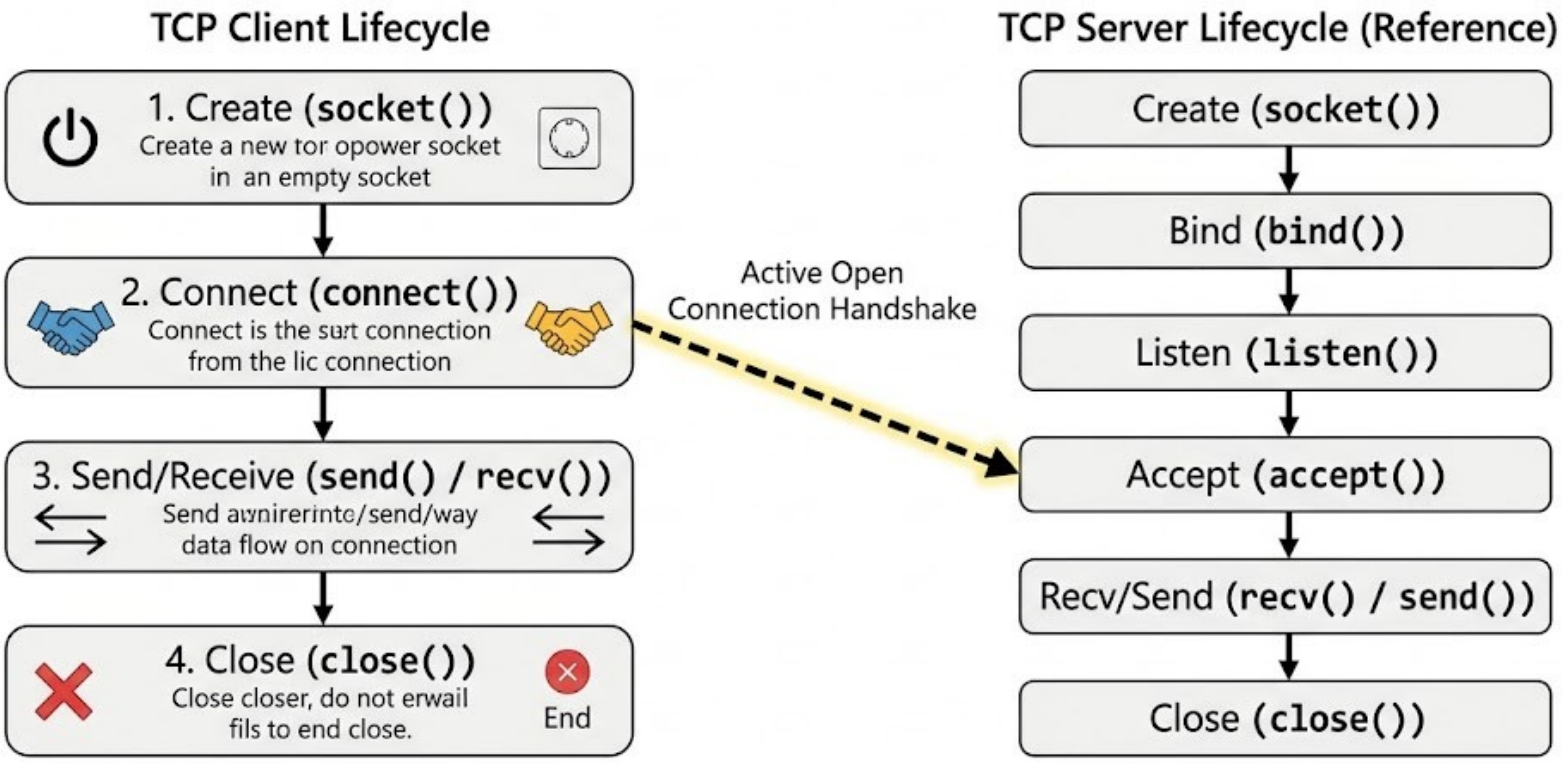
```
while True:
    data = conn.recv(1024)
    if not data: break # Client disconnected
    conn.sendall(data)
```

The TCP Client Flow

- The client is the instigator. It actively seeks out a listening server.

- Steps:
  - Create → Connect → Send/Receive → Close.

The TCP Client Flow



TCP Client: Connecting

- The host and port must exactly match what the server is bound to.
- .connect() establishes the TCP handshake.

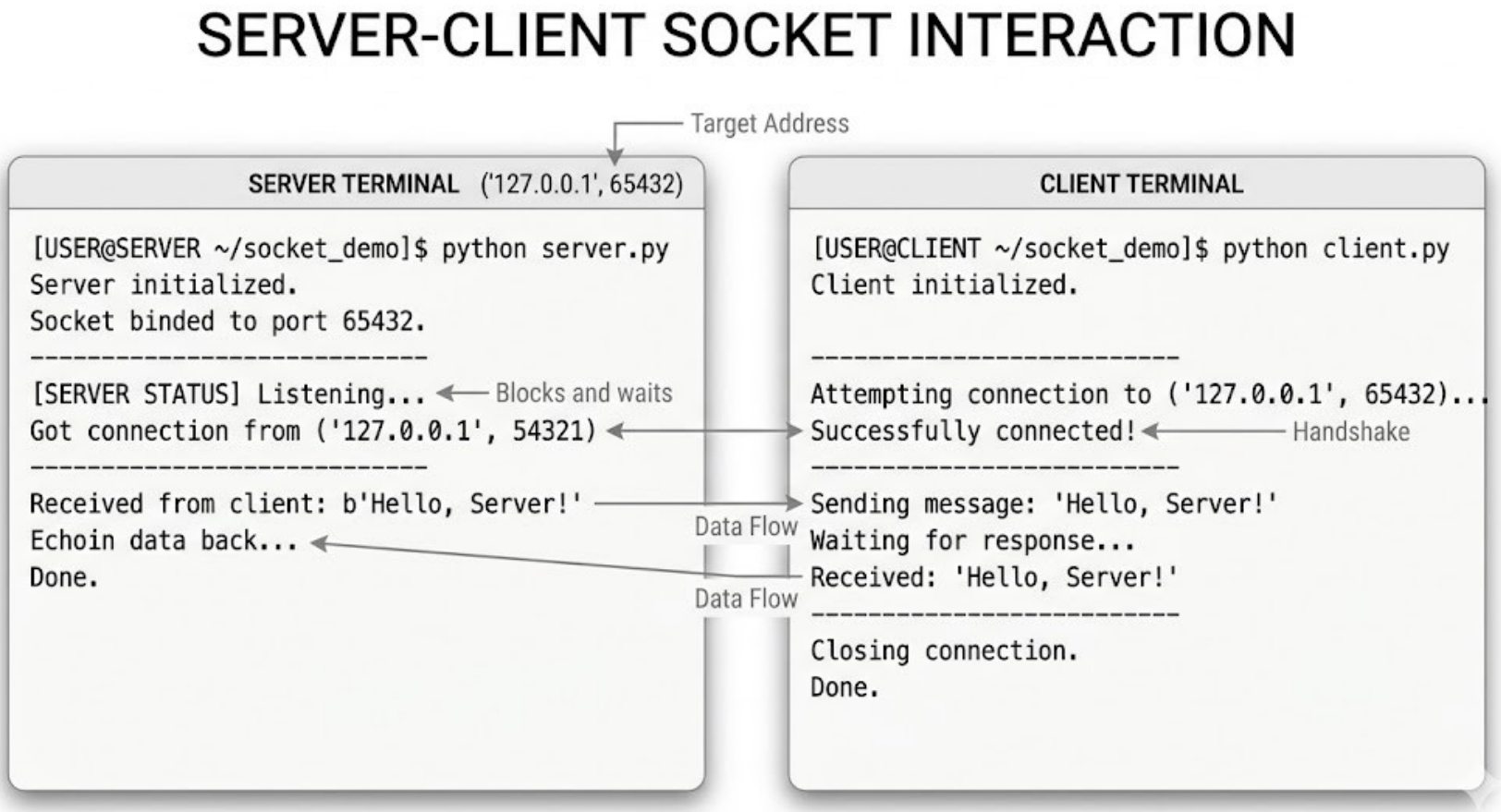
```
client_socket = socket.socket(socket.AF_INET,
socket.SOCK_STREAM)
client_socket.connect(('127.0.0.1', 65432))
```

TCP Client: Sending Data

- Data sent over sockets must be in bytes, not standard Python strings.
- Use .encode() before sending and .decode() after receiving.

```
message = "Hello, Server!"
client_socket.sendall(message.encode('utf-8'))
response = client_socket.recv(1024)
print(response.decode('utf-8'))
```

Running the TCP Echo Application



Running the TCP Echo Application

- You must run server.py first.
- Then, run client.py in a separate terminal.

Switching Gears to UDP

- No persistent connection (listen, accept, connect are skipped).
- Data is sent in independent chunks called "datagrams".
- Faster, but requires less overhead.

UDP Server setup

- SOCK_DGRAM specifies we are using UDP instead of TCP.
- We still bind to a port so the client knows where to send data.

```
# SOCK_DGRAM = UDP
server = socket.socket(socket.AF_INET,
socket.SOCK_DGRAM)
server.bind(('127.0.0.1', 54321))
```

UDP Server: Receive & Reply

- recvfrom() returns the data AND the address of the sender.
- sendto() requires you to specify the destination address every time.

```
while True:
    data, addr = server.recvfrom(1024)
    print(f"Message from {addr}")
    server.sendto(b"Received!", addr)
```

UDP Client: Fire and Forget

- The client skips connecting and just starts sending data to the target address.

```
client = socket.socket(socket.AF_INET,
socket.SOCK_DGRAM)
# Send directly to the address
client.sendto(b"Hello UDP", ('127.0.0.1', 54321))
data, server_addr = client.recvfrom(1024)
```

Best Practices: Bytes vs. Strings

- Networks do not understand Python strings. They only understand raw bytes.
- Always encode strings to UTF-8 before calling send() or sendall().
- Always decode bytes to strings after calling recv().

Best Practices: Resource Management

- Sockets tie up your operating system's ports.
- If your script crashes without closing the socket, the port stays blocked (often causing "Address already in use" errors).
- Use the with statement to ensure sockets close automatically.

```
# Automatically closes when the block ends
with socket.socket() as client_socket:
    client_socket.connect(...)
```

Best Practices: Expect Failure

- Network connections are inherently unreliable.
- Servers go down, Wi-Fi drops, and IP addresses change.
- Wrap network calls in try...except blocks.

```
try:
    client.connect((HOST, PORT))
except ConnectionRefusedError:
    print("Server is down. Try again later.")
```

Summary

- socket module is Python's gateway to networking.
- TCP for reliability (Web, Chat).
- UDP for speed (Video, Games).
- Always manage resources carefully and handle encoding.

Thank you

@mitu_skillologies

@mITuSkillologies

@mitu_group

@mitu-skillologies

@MITUSkillologies

kaggle

@mituskillologies

Web Resources
https://mitu.co.in
http://tusharkute.com

@mituskillologies

contact@mitu.co.in

tushar@tusharkute.com

Public Feedback